Кременчуцький національний університет імені Михайла Остроградського
(повне найменування вищого навчального закладу)

Кафедра комп’ютерної інженерії та електроніки
(повна назва кафедри)

ЛАБОРАТОРНА РОБОТА № 2

Тема: Скалярні типи даних, побітові операції та математичне моделювання

Мета роботи: Опанувати практичні навички роботи зі скалярними типами даних мови Python (int, float, complex, bool), вивчити внутрішні особливості представлення цілих чисел довільної точності та дійсних чисел стандарту IEEE 754. Навчитися реалізовувати складні аналітичні математичні моделі із застосуванням функцій стандартних модулів math і cmath, виконувати перетворення між різними позиційними системами числення, а також освоїти бітову арифметику та маніпуляції з бітовими масками для моделювання апаратних регістрів мікроконтролерів і низькорівневих телеметричних пакетів у задачах комп'ютерної інженерії.

Студента III курсу КІ-24-1 групи
Ступінь вищої освіти бакалавр
(бакалавр, магістр)
Спеціальність 123 – «Комп’ютерна інженерія»
Освітня програма Комп’ютерна інженерія
Яцентюка Євгенія Олександровича

м. Кременчук 2026 рік

1. Мета роботи

- Дослідити особливості роботи зі скалярними типами даних мови Python (`int`, `float`, `complex`, `bool`).
- Опанувати функції стандартних бібліотек `math` та `cmath` для аналітичного моделювання нелінійних функцій та розрахунку кіл змінного струму.
- Засвоїти практичні методи побітової обробки даних (оператори `&`, `|`, `^`, `~`, `<<`, `>>`), застосування бітових масок для пакування, розпакування та модифікації прапорців у пам'яті апаратних регістрів.

2. Постановка індивідуального завдання (Варіант 2)

Завдання 1. Скалярне математичне моделювання

1. Реалізувати обчислення аналітичної функції двох змінних $\mathrm{f(x,\ y)}$ за формулою:

$$f(x,y) = \frac{\ln(x^2+2)}{\cos(y-x)+1.5} + \sqrt[3]{x \cdot e^y}$$

2. Здійснити перевірку обчислювальної стійкості та збігу результатів з еталоном з урахуванням стандарту IEEE 754 за допомогою `math.isclose`

3. Виконати розрахунок комплексного імпедансу послідовного RLC-контуру:

$$Z = R + j \cdot \left(\omega L - \frac{1}{\omega C}\right), \quad \omega = 2\pi f$$

Визначити активний опір $\mathrm{Re(Z)}$, реактивний опір $\mathrm{Im(Z)}$, повний модуль імпедансу $|Z|$ та фазовий зсув $\arg(Z)$ у радіанах і градусах.

Завдання 2. Побітові операції та бітові маски

Спроєктувати систему пакування, верифікації розпакування та побітової маніпуляції 16-бітним регістром передавача згідно з картою бітових полів:

Назва поля	Бітовий діапазон	Розрядність	Допустимий діапазон	Опис
UART_BAUD	[15..12]	4 біти	$0 \dots 15 (0x0 \dots 0xF)$	Індекс швидкості передавача
TX_BUSY	[11]	1 біт	$0 \dots 1 (False/True)$	Прапорець зайнятості передавача
FIFO_CNT	[10..2]	9 бітів	$0 \dots 511 (0x000 \dots 0x1FF)$	Кількість байтів у черзі FIFO
MODE	[1..0]	2 біти	$0 \dots 3 (0x0 \dots 0x3)$	Режим роботи трансивера

Індивідуальна операція: Упакувати значення полів у 16-бітне число та виконати **скидання прапорця TX_BUSY (біт 11) у 0** без зміни стану інших полів.

3. Теоретичні відомості та алгоритмічний аналіз

1. **Представлення дійсних чисел:** Дійсні числа (float) реалізовані у форматі подвійної точності стандарту IEEE 754 (64 біти). Через наявність машинної похибки округлення пряме порівняння a == b є некоректним, тому перевірка рівності здійснюється через предикат math.isclose(a, b, rel_tol=1e-9).

2. **Комплексні числа:** Вбудований тип complex підтримує роботу з уявною одиницею j. Для переведення алгебраїчної форми $Z = R + jX$ у показникову (модуль та аргумент) застосовується функція cmath.polar(Z).

3. **Побітове пакування (Bit Packing):**

- Зсув поля ліворуч на відповідну позицію: $val \ll SHIFT$.
- Об'єднання через побітове «АБО»: $REG = field_1 \mid field_2 \mid \dots$

4. **Побітове розпакування (Bit Unpacking):**

- Зсув потрібного сегмента праворуч у молодші розряди:
- Накладання маски кон'юнкції «ТА»: $(REG \gg SHIFT)\ \&\ MASK$.

5.**Скидання окремого біта (Bit Clear):**

- Для скидання n-го біта формується інверсна маска: $\sim (1 \ll n)$.

- Виконується операція: $\mathrm{REG}_{\mathrm{new}} = \mathrm{REG}\ \&\ \sim(1 \ll n)$.

4. Лістинг програмного коду

4.1. Модуль 1: Скалярні обчислення (`scalar_modeling.py`)

```
 """
Лабораторна робота № 2. Модуль 1 (Варіант 2).
Аналітичне моделювання нелінійної функції двох змінних та розрахунок комплексного імпедансу.
Освітній компонент: Програмування на Python.
"""

import math
import cmath
import sys


def compute_variant_function(x: float, y: float) -> float:
  """
  Обчислює значення функції f(x, y) за формулою Варіанта 2:
  f(x, y) = ln(x^2 + 2) / (cos(y - x) + 1.5) + cbrt(x * e^y)
  """
  # 1. Чисельник першого доданка: ln(x^2 + 2)
  numerator = math.log(x**2 + 2.0)

  # 2. Знаменник першого доданка: cos(y - x) + 1.5
  # Оскільки cos(y - x) належить [-1; 1], знаменник завжди >= 0.5 > 0
  denominator = math.cos(y - x) + 1.5
  first_term = numerator / denominator

  # 3. Другий доданок: кубічний корінь з (x * e^y)
  radicand = x * math.exp(y)

  # Використовуємо math.cbrt (Python 3.11+) або універсальний розрахунок для дійсних чисел
  if hasattr(math, "cbrt"):
    second_term = math.cbrt(radicand)
  else:
    second_term = math.copysign(abs(radicand) ** (1.0 / 3.0), radicand)

  return first_term + second_term
```

```python
def compute_rlc_impedance(r_resistance: float, l_inductance: float, c_capacitance:
float, frequency_hz: float) -> complex:
    """
    Обчислює комплексний опір (імпеданс) послідовного RLC-контуру:
    Z = R + j * (omega * L - 1 / (omega * C)), де omega = 2 * pi * f
    """
    if frequency_hz <= 0.0 or c_capacitance <= 0.0 or l_inductance <= 0.0:
        raise ValueError("Параметри L, C та частота f повинні бути строго додатними.")

    omega = 2.0 * math.pi * frequency_hz
    reactance_l = omega * l_inductance
    reactance_c = 1.0 / (omega * c_capacitance)
    total_reactance = reactance_l - reactance_c

    return complex(r_resistance, total_reactance)


def main() -> None:
    """Головна точка входу для демонстрації аналітичних скалярних обчислень."""
    print("=" * 80)
    print(f"{'ВАРІАНТ 2: АНАЛІТИЧНЕ МОДЕЛЮВАННЯ ТА СКАЛЯРНІ ДАНІ':^80}")
    print("=" * 80)

    # 1. Розрахунок аналітичної функції f(x, y)
    test_x = 2.0
    test_y = 1.0
    result_val = compute_variant_function(test_x, test_y)

    # Еталонне значення для перевірки точності
    ref_first = math.log(test_x**2 + 2.0) / (math.cos(test_y - test_x) + 1.5)
    ref_arg = test_x * math.exp(test_y)
    ref_second = math.copysign(abs(ref_arg) ** (1.0 / 3.0), ref_arg)
    reference_val = ref_first + ref_second

    is_accurate = math.isclose(result_val, reference_val, rel_tol=1e-9,
abs_tol=1e-12)

    print(f"Вхідні змінні:                 x = {test_x:.4f}, y = {test_y:.4f}")
    print(f"Формула:                       f(x, y) = ln(x^2+2)/(cos(y-x)+1.5) +
cbrt(x*e^y)")
    print(f"Розраховане значення f(x, y):  {result_val:12.8f}")
    print(f"Еталонне значення верифікації: {reference_val:12.8f}")
    print(f"Точність IEEE 754 (isclose):   {str(is_accurate).upper()}
(rel_tol=1e-9)")
    print("-" * 80)
```

```
    # 2. Розрахунок комплексного імпедансу RLC контуру
    r_val = 120.0       # Ом
    l_val = 0.04        # Гн (40 мГн)
    c_val = 8.0e-6      # Ф (8 мкФ)
    freq_val = 250.0    # Гц

    z_complex = compute_rlc_impedance(r_val, l_val, c_val, freq_val)
    z_magnitude, z_phase_rad = cmath.polar(z_complex)
    z_phase_deg = math.degrees(z_phase_rad)

    print(f"{'РОЗРАХУНОК КОМПЛЕКСНОГО ІМПЕДАНСУ RLC-КОНТУРУ':^80}")
    print("-" * 80)
    print(f"Параметри: R = {r_val:.1f} Ом, L = {l_val * 1e3:.1f} мГн, C = {c_val *
1e6:.1f} мкФ, f = {freq_val:.1f} Гц")
    print(f"Комплексний імпеданс Z:         {z_complex.real:8.3f} +
{z_complex.imag:8.3f}j Ом")
    print(f"Активний опір Re(Z):            {z_complex.real:8.3f} Ом")
    print(f"Реактивний опір Im(Z):          {z_complex.imag:8.3f} Ом")
    print(f"Повний модуль імпедансу |Z|:    {z_magnitude:8.3f} Ом")
    print(f"Фазовий зсув (Arg Z):           {z_phase_deg:8.3f} град
({z_phase_rad:8.4f} рад)")
    print("=" * 80)


if __name__ == "__main__":
    main()
```

4.2. Модуль 2: Побітові операції над регістром (`register_bitmask.py`)

```
"""
Лабораторна робота № 2. Модуль 2 (Варіант 2).
Побітове упакування, розпакування та модифікація 16-бітного регістра передавача.
Поля: [15..12: UART_BAUD], [11: TX_BUSY], [10..2: FIFO_CNT], [1..0: MODE].
Операція варіанта: упакування полів та скидання прапорця TX_BUSY.
"""

import sys

# --- Бітові зсуви (Shift) та бітові маски (Mask) ---
# [15..12]: UART_BAUD — 4 біти (0b1111 = 0xF)
UART_BAUD_SHIFT = 12
UART_BAUD_MASK = 0xF

# [11]: TX_BUSY — 1 біт (0b1 = 0x1)
TX_BUSY_SHIFT = 11
TX_BUSY_MASK = 0x1
```

```
# [10..2]: FIFO_CNT — 9 бітів (0b1_1111_1111 = 0x1FF)
FIFO_CNT_SHIFT = 2
FIFO_CNT_MASK = 0x1FF

# [1..0]: MODE — 2 біти (0b11 = 0x3)
MODE_SHIFT = 0
MODE_MASK = 0x3


def pack_transmitter_register(uart_baud: int, tx_busy: bool, fifo_cnt: int, mode:
int) -> int:
  """
  Упаковує параметри передавача у 16-бітний регістр за допомогою
  операцій зсуву (<<) та побітового логічного АБО (|).
  """
  # Валідація діапазонів значень бітових полів
  if not (0 <= uart_baud <= UART_BAUD_MASK):
    raise ValueError(f"UART_BAUD виходить за межі 4 бітів (0..15): {uart_baud}")
  if not (0 <= fifo_cnt <= FIFO_CNT_MASK):
    raise ValueError(f"FIFO_CNT виходить за межі 9 бітів (0..511): {fifo_cnt}")
  if not (0 <= mode <= MODE_MASK):
    raise ValueError(f"MODE виходить за межі 2 бітів (0..3): {mode}")

  busy_bit = 1 if tx_busy else 0

  # Побітове складання
  register_val = (
    (uart_baud << UART_BAUD_SHIFT)
    | (busy_bit << TX_BUSY_SHIFT)
    | (fifo_cnt << FIFO_CNT_SHIFT)
    | (mode << MODE_SHIFT)
  )
  return register_val & 0xFFFF


def unpack_transmitter_register(register_val: int) -> dict:
  """
  Розпаковує 16-бітне число у словник значень окремих бітових полів
  за допомогою бітових зсувів праворуч (>>) та масок І (&).
  """
  uart_baud = (register_val >> UART_BAUD_SHIFT) & UART_BAUD_MASK
  tx_busy = bool((register_val >> TX_BUSY_SHIFT) & TX_BUSY_MASK)
  fifo_cnt = (register_val >> FIFO_CNT_SHIFT) & FIFO_CNT_MASK
  mode = (register_val >> MODE_SHIFT) & MODE_MASK

  return {
```

```
    "uart_baud": uart_baud,
    "tx_busy": tx_busy,
    "fifo_cnt": fifo_cnt,
    "mode": mode,
  }


def clear_tx_busy(register_val: int) -> int:
  """
  Індивідуальне завдання Варіанта 2:
  Скидає прапорець TX_BUSY (біт 11) у 0 за допомогою операторів & та інверсії ~.
  """
  mask = ~(1 << TX_BUSY_SHIFT)
  # Маскуємо до 16 біт (0xFFFF), щоб коректно врахувати інверсію в Python
  return (register_val & mask) & 0xFFFF


def set_tx_busy(register_val: int) -> int:
  """Встановлює прапорець TX_BUSY (біт 11) у 1 за допомогою оператора |."""
  mask = 1 << TX_BUSY_SHIFT
  return (register_val | mask) & 0xFFFF


def toggle_tx_busy(register_val: int) -> int:
  """Інвертує стан прапорця TX_BUSY (біт 11) за допомогою XOR (^)."""
  mask = 1 << TX_BUSY_SHIFT
  return (register_val ^ mask) & 0xFFFF


def main() -> None:
  """Головна точка входу для тестування бітових операцій Варіанта 2."""
  print("=" * 80)
  print(f"{'ВАРІАНТ 2: УПАКУВАННЯ ТА МАНІПУЛЯЦІЯ РЕГІСТРОМ ПЕРЕДАВАЧА':^80}")
  print("=" * 80)

  # Початкові тестові параметри
  baud_rate = 9        # Швидкість (0..15, наприклад 9 -> 115200 бод)
  busy_flag = True     # Прапорець зайнятості TX_BUSY = 1
  fifo_counter = 345   # Кількість байт у черзі (0..511)
  work_mode = 2        # Режим роботи (0..3)

  # 1. Упакування полів у 16-бітний регістр
  reg_word = pack_transmitter_register(baud_rate, busy_flag, fifo_counter,
work_mode)

  print(f"Вхідні дані:")
```

```
    print(f"  UART_BAUD = {baud_rate} (4 біти), TX_BUSY = {busy_flag} (біт 11),")
    print(f"  FIFO_CNT  = {fifo_counter} (9 бітів), MODE = {work_mode} (2 біти)")
    print("-" * 80)
    print(f"Упакований 16-бітний регістр:")
    print(f"  Десятковий формат (DEC): {reg_word}")
    print(f"  Шістнадцятковий (HEX):   0x{reg_word:04X}")
    print(f"  Двійковий формат (BIN):  0b{reg_word:016b}")
    print(f"  Вісімковий формат (OCT): 0o{reg_word:06o}")
    print("-" * 80)

    # 2. Розпакування та верифікація полів
    unpacked = unpack_transmitter_register(reg_word)
    print(f"{'ВЕРИФІКАЦІЯ РОЗПАКУВАННЯ':^80}")
    print(f"  UART_BAUD : {unpacked['uart_baud']} (очікувалося {baud_rate})")
    print(f"  TX_BUSY   : {unpacked['tx_busy']} (очікувалося {busy_flag})")
    print(f"  FIFO_CNT  : {unpacked['fifo_cnt']} (очікувалося {fifo_counter})")
    print(f"  MODE      : {unpacked['mode']} (очікувалося {work_mode})")
    print("-" * 80)

    # 3. Виконання основного завдання варіанта: Скидання прапорця TX_BUSY
    print(f"{'ВИКОНАННЯ ЗАВДАННЯ ВАРІАНТА: СКИДАННЯ TX_BUSY':^80}")
    reg_busy_cleared = clear_tx_busy(reg_word)
    unpacked_cleared = unpack_transmitter_register(reg_busy_cleared)

    print(f"  Стан до скидання:   0b{reg_word:016b} | TX_BUSY =
{unpacked['tx_busy']}")
    print(f"  Після CLEAR_BUSY:   0b{reg_busy_cleared:016b} | TX_BUSY =
{unpacked_cleared['tx_busy']}")
    print(f"  Нове значення HEX:  0x{reg_busy_cleared:04X}")
    print("-" * 80)

    # Додаткова перевірка встановлення та перемикання прапорця
    reg_busy_set = set_tx_busy(reg_busy_cleared)
    reg_busy_toggled = toggle_tx_busy(reg_busy_set)
    print(f"  Перевірка SET_BUSY:    0b{reg_busy_set:016b} | TX_BUSY =
{unpack_transmitter_register(reg_busy_set)['tx_busy']}")
    print(f"  Перевірка TOGGLE_BUSY: 0b{reg_busy_toggled:016b} | TX_BUSY =
{unpack_transmitter_register(reg_busy_toggled)['tx_busy']}")
    print("=" * 80)


if __name__ == "__main__":
    main()
```

5. Результати виконання програм

5.1. Виведення `scalar_modeling.py`

```
================================================================================
              ВАРІАНТ 2: АНАЛІТИЧНЕ МОДЕЛЮВАННЯ ТА СКАЛЯРНІ ДАНІ
================================================================================
Вхідні змінні:                  x = 2.0000, y = 1.0000
Формула:                        f(x, y) = ln(x^2+2)/(cos(y-x)+1.5) + cbrt(x*e^y)
Розраховане значення f(x, y):     2.63654480
Еталонне значення верифікації:    2.63654480
Точність IEEE 754 (isclose):    TRUE (rel_tol=1e-9)
--------------------------------------------------------------------------------
                РОЗРАХУНОК КОМПЛЕКСНОГО ІМПЕДАНСУ RLC-КОНТУРУ
--------------------------------------------------------------------------------
Параметри: R = 120.0 Ом, L = 40.0 мГн, C = 8.0 мкФ, f = 250.0 Гц
Комплексний імпеданс Z:          120.000 +  -16.746j Ом
Активний опір Re(Z):             120.000 Ом
Реактивний опір Im(Z):           -16.746 Ом
Повний модуль імпедансу |Z|:     121.163 Ом
Фазовий зсув (Arg Z):             -7.944 град ( -0.1387 рад)
================================================================================
```

5.2. Виведення `register_bitmask.py`

```
================================================================================
           ВАРІАНТ 2: УПАКУВАННЯ ТА МАНІПУЛЯЦІЯ РЕГІСТРОМ ПЕРЕДАВАЧА
================================================================================
Вхідні дані:
  UART_BAUD = 9 (4 біти), TX_BUSY = True (біт 11),
  FIFO_CNT  = 345 (9 бітів), MODE = 2 (2 біти)
--------------------------------------------------------------------------------
Упакований 16-бітний регістр:
  Десятковий формат (DEC): 40294
  Шістнадцятковий (HEX):   0x9D66
  Двійковий формат (BIN):  0b1001110101100110
  Вісімковий формат (OCT): 0o116546
--------------------------------------------------------------------------------
                            ВЕРИФІКАЦІЯ РОЗПАКУВАННЯ
  UART_BAUD : 9 (очікувалося 9)
  TX_BUSY   : True (очікувалося True)
  FIFO_CNT  : 345 (очікувалося 345)
  MODE      : 2 (очікувалося 2)
--------------------------------------------------------------------------------
                ВИКОНАННЯ ЗАВДАННЯ ВАРІАНТА: СКИДАННЯ TX_BUSY
  Стан до скидання:   0b1001110101100110 | TX_BUSY = True
  Після CLEAR_BUSY:   0b1001010101100110 | TX_BUSY = False
  Нове значення HEX:  0x9566
--------------------------------------------------------------------------------
  Перевірка SET_BUSY:    0b1001110101100110 | TX_BUSY = True
  Перевірка TOGGLE_BUSY: 0b1001010101100110 | TX_BUSY = False
================================================================================
```

6. Аналітична верифікація та обговорення результатів

6.1. Аналітичний розрахунок функції $\mathrm{f(x,\ y)x = 2.0,y = 1.0}$

1. Чисельник першого доданка:

$\ln(2^2 + 2) = \ln(6) \approx 1.79175947$

2. Знаменник першого доданка:

$\cos(1.0 - 2.0) + 1.5 = \cos(-1.0) + 1.5 \approx 0.54030231 + 1.5 = 2.04030231$

3. Перший доданок:

$\frac{1.79175947}{2.04030231} \approx 0.87818331$

4. Другий доданок:

$\sqrt[3]{2.0 \cdot e^{1.0}} = \sqrt[3]{2 \cdot 2.71828183} = \sqrt[3]{5.43656366} \approx 1.75836149$

5. Підсумок:

$f(2, 1) = 0.87818331 + 1.75836149 = 2.63654480$

Розрахунок програми повністю збігся з аналітичним результатом із точністю до 10^{-8}.

6.2. Розрахунок параметрів RLC-контуру

$\omega = 2\pi \cdot 250 = 500\pi \approx 1570.7963\ /$

- Індуктивний опір: $\mathrm{X}_L = \omega L = 1570.7963 \cdot 0.04 = 62.832$
- Ємнісний опір: $\mathrm{X}_C = \frac{1}{\omega C} = \frac{1}{1570.7963 \cdot 8 \cdot 10^{-6}} = 79.577$
- Реактивний опір: $\mathrm{X} = \mathrm{X}_L - X_C = 62.832 - 79.577 = -16.745$
- Повний модуль імпедансу:

$|Z| = \sqrt{R^2 + X^2} = \sqrt{120.0^2 + (-16.745)^2} = \sqrt{14400 + 280.42} \approx 121.163$

- Фазовий кут:

$\varphi = \mathrm{atan2}(-16.745, 120.0) \approx -0.1387\ \approx -7.944°$

Оскільки $\mathrm{X}_C > X_L\ (X < 0)$, коло має ємнісний характер, а фаза струму випереджає фазу напруги.

6.3. Аналіз побітової карти регістра

Вхідні значення у двійковому форматі:

- `UART_BAUD` = 9 $\rightarrow 1001_2$
- `TX_BUSY` = 1 $\rightarrow 1_2$
- `FIFO_CNT` = 345 $\rightarrow 101011001_2$
- `MODE` = 2 $\rightarrow 10_2$
    - Об'єднання у 16-бітне слово:

$$\underbrace{1001}_{[15..12]}\ \underbrace{1}_{[11]}\ \underbrace{101011001}_{[10..2]}\ \underbrace{10}_{[1..0]} = 1001\,1101\,0110\,0110_2 = 0\text{x}9\text{D}66 = 40294_{10}$$

    - Скидання прапорця `TX_BUSY` (біт 11):
- Маска скидання:$\sim (1 \ll 11) = \sim (0\text{x}0800) = 0\text{x}\text{F}7\text{FF}$
- $0\text{x}9\text{D}66\ \ \&\ \ 0\text{x}\text{F}7\text{FF} = 0\text{x}9566$
- Двійкове представлення: $1001\,0101\,0110\,0110_2$. Біт 11 гарантовано переведено в 0, сусідні поля збережено без змін.

7. Висновки

1. У ході виконання лабораторної роботи було досліджено математичні бібліотеки Python (`math`, `cmath`) для скалярного моделювання.

   Здійснено порівняння чисел з плаваючою крапкою за допомогою функції math.isclose(), що підтвердило стабільність обчислень у межах допустимих похибок стандарту IEEE 754.

2. Проведено розрахунок комплексного опору послідовного RLC-контуру з використанням вбудованого типу `complex` та модуля `cmath`. Визначено, що при заданих параметрах контур має ємнісний характер реактивного опору ($\text{Im}(\text{Z}) \approx -16.746\text{Om}$) з від'ємним фазовим зсувом -7.944°.

3. Реалізовано алгоритми пакування та розпакування бітових полів апаратного 16-бітного регістра передавача за допомогою зсувів <<, >> та логічних операторів `|`, &.

4. Виконано індивідуальне завдання Варіанта 2: реалізовано скидання біта стану `TX_BUSY` (біт 11) за допомогою інверсної бітової маски $\sim (1 \ll 11)$, що перевело регістр зі стану `0x9D66` у стан `0x9566` без спотворення інформації у суміжних бітових полях. Результати верифіковано консольними тестами.

Контрольні питання:

1. Яким чином у CPython реалізовано довгу арифметику для типу int? Чому цілі числа в Python не мають фіксованого діапазону переповнення розрядної сітки ($2^{31} - 1$ або $2^{63} - 1$), властивого мовам C/C++?

У CPython тип `int` реалізовано на рівні мови C через структуру `PyLongObject`. Вона містить поле `ob_size`, що визначає кількість елементів у масиві та знак числа, а також динамічний масив «цифр» `digit ob_digit[]`.

- На 64-бітних платформах як «цифра» використовується 2^{30}-кова система числення (30 інформаційних бітів у кожному 32-бітному машинному слові), що залишає вільні біти для швидкої обробки переносу (carry).
- На відміну від мов C/C++, де типи прив'язані до фіксованих апаратних регістрів процесора (32 або 64 біти) і переповнюються апаратно, у Python пам'ять під масив розрядів виділяється динамічно в оперативній пам'яті (купі). Тому розмір числа обмежений лише загальним обсягом доступної оперативної пам'яті комп'ютера.

2. Опишіть структуру 64-бітного представлення дійсного числа float за стандартом IEEE 754. Чому виникає похибка представлення числа 0.1 у двійковій системі та як правильно виконувати порівняння чисел за допомогою функції math.isclose()?

Згідно зі стандартом IEEE 754 (binary64 / double precision), 64-бітне число складається з трьох полів:

1. **Знак (S, Sign)** — 1 біт (біт 63): `0` — додатне, `1` — від'ємне.
2. **Порядок (E, Exponent)** — 11 бітів (біти 62..52): зміщений порядок зі зміщенням $+1023$.
3. **Мантиса (M, Mantissa/Fraction)** — 52 біти (біти 51..0): дробова частина з нормалізованою неявною одиницею ($1.\mathrm{M}$), що забезпечує 53 біти точності ($\approx 15 \ldots 17$ десяткових знаків).

   $\mathrm{Value} = (-1)^S \cdot (1 + M) \cdot 2^{E-1023}$

- **Похибка 0.1:** Дріб є скінченним у двійковій системі числення лише тоді, коли його знаменник розкладається виключно на степені двійки (2^k). Число $0.1 = 1_{\overline{10=\frac{1}{2\cdot 5}}}$ через наявність множника 5 перетворюється на **нескінченний періодичний дріб**: $0.0\overline{0011}_2$. Оскільки розрядна сітка мантиси обмежена 52 бітами, дріб усікається/округлюється, що породжує похибку машинного представлення ($\approx 0.1000000000000000555$).
- **Порівняння через `math.isclose()`:** Пряме порівняння `a == b` є неприпустимим. Функція `math.isclose(a, b, rel_tol=1e-9, abs_tol=0.0)` перевіряє умову$|a - b| \leq \max(\mathrm{rel_tol} \cdot \max(|a|, |b|), \mathrm{abs_tol})$, де `rel_tol` контролює збіг значущих цифр незалежно від масштабу, а `abs_tol` запобігає збоям порівняння при значеннях, близьких до нуля.

3. Поясніть фізичний і математичний зміст операції побітового інвертування `~x`. Чому вираз `~0` у Python дорівнює `-1`, а `~0x5B29` вимагає додаткового маскування `& 0xFFFF` для обмеження 16-бітної розрядності?

Зміст `~x`:

- *Фізичний:* порозрядна логічна інверсія NOT кожного біта ($0 \to 1,\ 1 \to 0$).

- *Математичний:* у системі доповняльного коду знакове інвертування описується формулою $\sim x = -x - 1$.

- **Чому `~0 == -1`:** Число 0 у бітовому представленні складається з нулів (`...0000_2`). Інвертування перетворює всі розряди на одиниці (`...1111_2`), що у доповняльному коді відповідає значенню -1 ($\sim 0 = -0 - 1 = -1$).

- **Маскування `~0x5B29 & 0xFFFF`:** Оскільки в Python цілі числа мають динамічну довжину, знаковий розряд фіктивно поширюється вліво нескінченно. Вираз ~0x5B29 перетворюється на від'ємне число `-(0x5B29 + 1) = -23338` із нескінченним рядом старших одиниць. Щоб відсікти ці знакові розряди та отримати коректне 16-бітне додатне беззнакове число, застосовують маску кон'юнкції `& 0xFFFF`, що залишає лише молодші 16 бітів (`0xA4D6`).

4. Які побітові операції та маски слід застосувати для встановлення в одиницю, скидання в нуль та інвертування k-го біта апаратного регістра без зміни стану решти бітів?

Формується базова одинична маска: `mask = (1 << k)`. **Встановлення в одиницю (Set):**

- Вираз: `reg = reg | (1 << k)` (оператор побітового «АБО» |).
- Властивість: $\mathrm{x} \mid 1 = 1,\ \ x \mid 0 = x$ (решта бітів залишаються незмінними).

- **Скидання в нуль (Clear):**

- Вираз: `reg = reg & ~(1 << k)` (побітове «ТА» & з інверсною маскою).
- Властивість:$\mathrm{x}\ \ \&\ \ 0 = 0$,$\mathrm{x}\ \ \&\ \ 1 = \mathrm{x}$ (k-й біт занулюється, решта зберігаються).

- **Інвертування (Toggle):**

- Вираз: `reg = reg ^ (1 << k)` (побітове виключне «АБО» ^, XOR).
- Властивість: x $\oplus 1 = \neg x$, $x \oplus 0 = x$(значення k-го біта змінюється на протилежне).

5. Чим відрізняється робота математичних функцій модуля `math` від модуля `cmath` при спробі обчислення квадратного кореня з від'ємного числа $\sqrt{-4}$?

- **Модуль `math`:** Спроєктований для роботи виключно в межах множини дійсних чисел (R). Оскільки операція $\sqrt{-4}$ не визначена на дійсній прямій, виклик `math.sqrt(-4)` генерує аварійне виключення `ValueError: math domain error`.
- **Модуль `cmath`:** Працює на множині комплексних чисел (C). Функція `cmath.sqrt(-4)` коректно повертає комплексне число з нульовою дійсною та ненульовою уявною частиною: `2j` (еквівалентно `0.0 + 2.0j`).

6. Виведіть загальну формулу упакування трьох цілочисельних параметрів A (розрядність 4 біти), B (розрядність 6 бітів) та C (розрядність 6 бітів) у єдине 16-бітне машинне слово. Які маски необхідно застосувати для зворотного розпакування поля B?

Розподіл бітової карти регістра (сума розрядів: $4 + 6 + 6 = 16$):

- Поле A [15..12]: 4 біти, зсув $\mathrm{SHIFT}_A = 12$, маска `0xF` (`0b1111`)
- Поле B [11..6]: 6 бітів, зсув $\mathrm{SHIFT}_B = 6$, маска `0x3F` (`0b11_1111`)
- ПолеC [5..0]: 6 бітів, зсув $\mathrm{SHIFT}_C = 0$, маска `0x3F` (`0b11_1111`)

- **Загальна формула упакування у 16-бітне слово W:**

$W = ((A \ \& \ 0\mathrm{xF}) \ll 12) \mid ((B \ \& \ 0\mathrm{x3F}) \ll 6) \mid (C \ \& \ 0\mathrm{x3F})$

- **Розпакування поля B:**

- **Спосіб 1 (зсув праворуч + маска молодших розрядів):**

  Зсуваємо слово праворуч на 6 бітів і накладаємо 6-бітну маску 0x3F:

  $B = (W \gg 6)\ \&\ 0\text{x}3\text{F}$

- **Спосіб 2 (маскування на місці + зсув праворуч):**

  Накладаємо маску розрядів [11..6], яка дорівнює $(0\text{x}3\text{F} \ll 6) = 0\text{x}0\text{FC}0$ (`0b0000_1111_1100_0000`), після чого зсуваємо результат:

  $B = (W\ \&\ 0\text{x}0\text{FC}0) \gg 6$